

Dossier Professionnel

Candidat : Vladimir Rodzaevskiy **Date de naissance** : 31 mai 1982 **Titre Professionnel visé** : Développeur Web et Web Mobile (DWWM) **Niveau** : 5 (équivalent Bac +2) **Code RNCP** : 37674 **Centre de formation** : AFPA Marseille **Année** : 2026

Sommaire

- [1. Présentation du parcours](#)
 - [2. Activité-Type n°1 — Développer la partie front-end](#)
 - Exemple 1.1 : Authentification & session avec contextes React
 - Exemple 1.2 : Vue calendrier mensuelle responsive
 - Exemple 1.3 : Intégration QR code & abonnement webcal
 - [3. Activité-Type n°2 — Développer la partie back-end](#)
 - Exemple 2.1 : Authentification JWT + bcrypt + middlewares composables
 - Exemple 2.2 : CRUD tâches avec récurrence et validation parent
 - Exemple 2.3 : Génération de flux iCalendar (RFC 5545)
 - [4. Projet professionnel](#)
 - [5. Annexes](#)
-

1. Présentation du parcours

1.1 Parcours académique

Année	Diplôme	Pays
2004	Master 2 Économie et Gestion d'Entreprise	Russie — Moscou
2012	Master 2 Commerce International	France
2026	Titre Professionnel DWWM (en cours)	France — AFPA Marseille

1.2 Parcours professionnel

Période	Poste	Secteur
~10 ans	Manager + serveur de salle	Restauration
3 ans	Croupier	Casino
2026	Reconversion en cours	Développement web

1.3 Motivations de la reconversion

Après une longue expérience dans les métiers de service à la personne (restauration et casino), j'ai souhaité me réorienter vers un métier qui combine plusieurs aspects qui me sont importants :

- Réflexion structurée** — la résolution de problèmes algorithmiques s'apparente à la logique économique que j'ai étudiée en Master.
- Création concrète** — voir un produit qu'on a construit utilisé par d'autres procure une satisfaction comparable à un service salle réussi.

- **Apprentissage continu** — l'écosystème du développement évolue rapidement, ce qui correspond à mon goût pour l'autoformation.
- **Conditions de travail** — souhait d'un métier moins physique et permettant plus de flexibilité horaire.

Mes deux Masters en économie m'ont donné une rigueur analytique que je retrouve dans la conception de logiciels (modélisation, abstraction, mesure des compromis). Mes années en salle de restaurant et en casino m'ont appris la **gestion sous pression**, la **rigueur des process** et la **communication claire** avec des publics variés — autant de qualités transposables dans une équipe de développement.

1.4 Compétences transversales acquises hors développement

Domaine	Apport pour le développement
Gestion d'équipe (manager)	Capacité à coordonner, déléguer, rendre compte
Service client (serveur)	Empathie utilisateur, sens du détail UX
Croupier (jeu, calculs rapides)	Rigueur, gestion du stress, attention soutenue
Master Économie et Gestion d'Entreprise (Moscou 2004)	Modélisation, raisonnement analytique, lecture critique
Master 2 Commerce International (France 2012)	Compréhension des enjeux produit / marché, intégration culturelle

2. Activité-Type n°1 — Développer la partie front-end d'une application web

Description de l'activité-type

(Référentiel REAC DWWM — RNCP 37674)

Le développeur web et web mobile maquette et développe l'interface utilisateur d'une application web ou web mobile en utilisant les langages, frameworks et bibliothèques adaptés. Il intègre les recommandations de sécurité, d'accessibilité et d'éco-conception. Il vérifie le bon fonctionnement de l'interface développée.

Compétences couvertes par mes exemples

- C1.1 — Maquetter une application
- C1.2 — Réaliser une interface utilisateur web statique et adaptable
- C1.3 — Développer une interface utilisateur web dynamique
- C1.4 — Réaliser une interface utilisateur avec une solution de gestion de contenu ou un framework

♦ Exemple 1.1 — Authentification et session avec contextes React

Projet : Reminder Famille **Période** : Avril-Mai 2026 **Stack** : React 18, React Router 6, Context API, fetch (AJAX)

Contexte

L'application est multi-utilisateurs et nécessite une gestion fine de la session : connexion, déconnexion, persistance du token JWT au reload, protection des routes privées, et accès à l'utilisateur courant depuis n'importe quel composant.

J'ai conçu un **système d'authentification basé sur deux contextes React** (`AuthContext` et `FamilyContext`) et un composant `ProtectedRoute` qui redirige vers `/login` si l'utilisateur n'est pas connecté.

Réalisation

1. Stockage du token au login

```
// front/src/context/AuthContext.jsx (simplifié)
import { createContext, useContext, useState, useEffect } from 'react';
import { authApi } from '../api/auth.api';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  // Récupération du user au reload si token présent
  useEffect(() => {
    const token = localStorage.getItem('reminder_token');
    if (!token) { setLoading(false); return; }
    authApi.me()
      .then(setUser)
      .catch(() => localStorage.removeItem('reminder_token'))
      .finally(() => setLoading(false));
  }, []);

  async function login(email, password) {
    const { token, user } = await authApi.login({ email, password });
    localStorage.setItem('reminder_token', token);
    setUser(user);
  }

  function logout() {
    localStorage.removeItem('reminder_token');
    setUser(null);
  }

  return (
    <AuthContext.Provider value={{ user, login, logout, loading }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

2. Protection des routes

```
// front/src/components/layout/ProtectedRoute.jsx
import { Navigate } from 'react-router-dom';
import { useAuth } from '../../../context/AuthContext';

export function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();
  if (loading) return <div>Chargement...</div>;
  if (!user) return <Navigate to="/login" replace />;
  return children;
}
```

3. Câblage dans App.jsx

```
<Routes>
  <Route path="/login" element={<Login />} />
  <Route path="/dashboard" element={
    <ProtectedRoute><Dashboard /></ProtectedRoute>
  } />
  { /* ... */ }
</Routes>
```

Compétences mises en œuvre

- Gestion d'état global avec **React Context API**
- Effet de bord contrôlé (`useEffect`) pour réhydrater la session
- Routage déclaratif avec **React Router 6**
- Persistance via `localStorage` avec gestion des erreurs (token expiré → nettoyage automatique)
- Pattern « hook personnalisé » (`useAuth`) pour exposer l'API

Résultat

L'authentification est totalement transparente pour l'utilisateur : il ferme l'onglet, revient le lendemain, il est toujours connecté tant que le JWT (durée 7 jours) reste valide. Les routes privées sont protégées sans condition de race ni flash de contenu non autorisé.

Référence dans le code source

- [front/src/context/AuthContext.jsx](#)
- [front/src/components/layout/ProtectedRoute.jsx](#)
- [front/src/App.jsx](#)

♦ Exemple 1.2 — Vue calendrier mensuelle responsive

Projet : Reminder Famille **Période** : Mai 2026 **Stack** : React 18, CSS Grid, JavaScript natif (Date API)

Contexte

Les utilisateurs voulaient visualiser leurs tâches non pas seulement sous forme de liste, mais aussi dans un **calendrier mensuel** type Google Calendar : chaque jour montre les tâches qui y tombent, avec des points de couleur par priorité.

J'ai conçu une vue calendrier **sans bibliothèque externe** pour rester en contrôle total du rendu, de la navigation et du responsive.

Réalisation

1. Calcul de la grille du mois

```
// front/src/pages/CalendarView.jsx (extrait)
function buildMonthGrid(year, month) {
  const firstDay = new Date(year, month, 1);
  const lastDay = new Date(year, month + 1, 0);
  const startWeekday = (firstDay.getDay() + 6) % 7; // lundi = 0
  const daysInMonth = lastDay.getDate();

  const cells = [];
  // cellules vides avant le 1er du mois
  for (let i = 0; i < startWeekday; i++) cells.push(null);
  for (let d = 1; d <= daysInMonth; d++) {
    cells.push(new Date(year, month, d));
  }
  // compléter pour avoir des semaines complètes
  while (cells.length % 7 !== 0) cells.push(null);
  return cells;
}
```

2. Affichage avec CSS Grid

```
return (
  <div className="calendar-grid">
    {[ 'Lun', 'Mar', 'Mer', 'Jeu', 'Ven', 'Sam', 'Dim' ].map(d =>
      <div className="calendar-header">{d}</div>
    )}
    {cells.map((date, i) =>
      <CalendarCell
        key={i}
        date={date}
        tasks={date ? tasksByDate[isoDate(date)] || [] : []}
        onClick={() => date && setSelectedDate(date)}
      />
    )}
  </div>
);
```

```
.calendar-grid {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  gap: 4px;
}

.calendar-cell {
  aspect-ratio: 1;          /* cases carrées */
  padding: 0.5rem;
  display: flex;
```

```

    flex-direction: column;
  }

  .priority-dot {
    width: 6px; height: 6px;
    border-radius: 50%;
  }
  .priority-dot.high { background: var(--priority-high); }
  .priority-dot.medium { background: var(--priority-medium); }
  .priority-dot.low { background: var(--priority-low); }

  @media (max-width: 600px) {
    .calendar-cell { padding: 0.25rem; font-size: 0.75rem; }
  }

```

3. Navigation mois précédent/suivant

```

function goToMonth(delta) {
  setCurrentDate(prev => {
    const d = new Date(prev);
    d.setMonth(d.getMonth() + delta);
    return d;
  });
}

```

Compétences mises en œuvre

- Manipulation de l'**API Date** native (sans bibliothèque type moment.js)
- **CSS Grid** avec aspect-ratio pour cases carrées
- Memoization des calculs (`useMemo` sur la grille pour éviter le recalcul à chaque render)
- **Responsive design** mobile-first avec media queries
- Gestion d'état local (mois affiché, jour sélectionné)

Résultat

Une vue calendrier interactive, performante (rendu < 50ms même sur mobile), responsive du smartphone au desktop, et **zéro dépendance externe** ajoutée au bundle.

Référence dans le code source

- [front/src/pages/CalendarView.jsx](#)

♦ Exemple 1.3 — Intégration QR code et abonnement webcal

Projet : Reminder Famille **Période** : Mai 2026 **Stack** : React 18, bibliothèque `qrcode` , API Canvas, scheme webcal://

Contexte

L'application expose un flux iCalendar permettant à l'utilisateur de recevoir les rappels de tâches dans le calendrier natif de son téléphone. Pour simplifier l'abonnement, j'ai conçu une **page Profil affichant un QR code** qui, scanné avec un téléphone, ouvre directement l'application calendrier en mode abonnement.

Réalisation

1. Génération du QR code dans React

```
// front/src/pages/Profile.jsx (extrait)
import { useEffect, useRef } from 'react';
import QRCode from 'qrcode';

function CalendarQR({ token, label, sub = '' }) {
  const canvasRef = useRef(null);

  useEffect(() => {
    // webcal:// au lieu de http:// = ouverture directe dans Calendrier
    const url = `webcal://${window.location.host.replace(':5173', ':3000')}` +
      `/api/calendar/${token}${sub}`;
    QRCode.toCanvas(canvasRef.current, url, {
      width: 200,
      margin: 1,
      color: { dark: '#1e1b3a', light: '#ffffff' }
    });
  }, [token, sub]);

  return (
    <div className="qr-block">
      <h3>{label}</h3>
      <canvas ref={canvasRef} />
      <a href={`webcal://...`}>📅 Ouvrir dans Calendrier</a>
    </div>
  );
}
```

2. Trois flux disponibles

```
<CalendarQR token={user.calendar_token}
  label="Tout mon calendrier" />
<CalendarQR token={user.calendar_token}
  label="Mes tâches personnelles"
  sub="/perso.ics" />
{families.map(f =>
  <CalendarQR token={user.calendar_token}
    label={`Famille ${f.name}`}
    sub={` /family/${f.id}.ics`} />
)}
```

Compétences mises en œuvre

- Intégration de **bibliothèque tierce** (qrcode) via npm
- Usage de l'**API Canvas** (référence DOM via useRef)
- Effet de bord avec **useEffect** et dépendances ciblées
- Manipulation d'URL et schémas custom (webcal://)
- **Composition de composants** (1 composant générique, 3 usages)

Résultat

L'utilisateur scanne 1 QR avec son iPhone ou Android, confirme l'abonnement, et reçoit ensuite ses rappels en notifications système sans jamais avoir à rouvrir l'application web. Testé sur iOS 17 et Android 14.

Référence dans le code source

- [front/src/pages/Profile.jsx](#)

3. Activité-Type n°2 — Développer la partie back-end d'une application web

Description de l'activité-type

(Référentiel REAC DWWM — RNCP 37674)

Le développeur web et web mobile crée, modifie, fait évoluer une base de données, en développe les composants d'accès aux données. Il développe la partie back-end d'une application web ou web mobile en appliquant les recommandations de sécurité, d'éco-conception et de performance. Il s'assure du bon fonctionnement par des tests appropriés.

Compétences couvertes par mes exemples

- C2.1 — Créer une base de données
- C2.2 — Développer les composants d'accès aux données
- C2.3 — Développer la partie back-end d'une application web
- C2.4 — Élaborer et mettre en œuvre des composants dans une application de gestion de contenu ou e-commerce

♦ Exemple 2.1 — Authentification JWT + bcrypt + middlewares composables

Projet : Reminder Famille **Période** : Avril–Mai 2026 **Stack** : Node.js 22, Express 4, jsonwebtoken, bcrypt, mysql2

Contexte

L'application multi-utilisateurs nécessite un système d'authentification sécurisé et un mécanisme d'autorisation **composable** : selon la route, on peut avoir besoin d'exiger « connecté », « membre d'une famille », « membre actif d'une famille », ou « administrateur d'une famille ».

Plutôt que de dupliquer ces vérifications dans chaque controller, j'ai conçu une **chaîne de middlewares** combinables.

Réalisation

1. Hachage bcrypt à l'inscription

```
// back/src/services/password.service.js
import bcrypt from 'bcrypt';

const COST = 10; // ~100ms par hash sur CPU moderne

export async function hashPassword(plain) {
  return bcrypt.hash(plain, COST);
}
```

```

}

export async function comparePassword(plain, hash) {
  return bcrypt.compare(plain, hash);
}

```

2. Signature JWT au login

```

// back/src/services/jwt.service.js
import jwt from 'jsonwebtoken';
import { env } from '../config/env.js';

export function signToken(userId) {
  return jwt.sign({ sub: userId }, env.jwt.secret, {
    expiresIn: env.jwt.expiresIn, // 7d
    algorithm: 'HS256',
  });
}

export function verifyToken(token) {
  return jwt.verify(token, env.jwt.secret);
}

```

3. Middleware d'authentification

```

// back/src/middleware/auth.js
import { verifyToken } from '../services/jwt.service.js';
import { unauthorized } from '../utils/httpError.js';

export function authRequired(req, res, next) {
  const auth = req.headers.authorization || '';
  const [scheme, token] = auth.split(' ');
  if (scheme !== 'Bearer' || !token) {
    throw unauthorized('Non authentifié');
  }
  try {
    const { sub } = verifyToken(token);
    req.user = { id: sub };
    next();
  } catch (err) {
    throw unauthorized('Token invalide ou expiré');
  }
}

```

4. Middlewares d'autorisation famille (composables)

```

// back/src/middleware/familyAccess.js
export async function requireFamilyMember(req, res, next) {
  const familyId = Number(req.params.familyId);
  const member = await familyMemberModel.findByIdAndUser(familyId, req.user.id);
}

```

```

    if (!member) throw forbidden('Pas membre de cette famille');
    req.familyMember = member;
    next();
  }

  export function requireAdmin(req, res, next) {
    if (!req.familyMember?.is_admin) throw forbidden('Action réservée aux administrateurs');
    next();
  }
}

```

5. Chaînage déclaratif dans les routes

```

// back/src/routes/families.routes.js
router.post('/:familyId/regenerate-code',
  asyncHandler(requireFamilyMember),
  requireAdmin,
  asyncHandler(familiesController.regenerateCode));

```

Compétences mises en œuvre

- **bcrypt** pour le hachage (coût 10, résistant aux GPU)
- **JWT HS256** signé avec secret stocké en `.env` (jamais en dur)
- Pattern **middlewares Express composables** (chaque middleware fait UNE chose)
- Gestion d'erreurs cohérente avec exceptions typées (`unauthorized` , `forbidden`)
- Récupération du contexte utilisateur depuis `req.user`

Résultat

Sécurité au niveau état de l'art pour ce type d'application. Le test d'intégration `tests/integration.sh` couvre 11 scénarios d'authentification (token absent, invalide, expiré, mauvais mot de passe, etc.) — tous passent.

Référence dans le code source

- [back/src/middleware/auth.js](#)
- [back/src/middleware/familyAccess.js](#)
- [back/src/services/jwt.service.js](#)
- [back/src/services/password.service.js](#)

♦ Exemple 2.2 — CRUD tâches avec récurrence et validation parent

Projet : Reminder Famille **Période** : Mai 2026 **Stack** : Node.js 22, Express, mysql2 (requêtes préparées), ENUM MySQL

Contexte

Les tâches sont l'entité métier centrale. Elles peuvent être personnelles ou familiales, assignées ou non à un membre précis, ponctuelles (`once`) ou récurrentes (`daily/weekly/monthly/yearly`).

J'ai implémenté un **CRUD complet** avec :

- Permissions différenciées selon rôle (parent vs enfant)
- Avancement automatique de l'échéance pour tâches récurrentes

- Workflow de validation parent (pending → pending_review → completed)

Réalisation

1. Modèle SQL pur (sans ORM)

```
// back/src/models/task.model.js (extrait)
import { pool } from '../db/pool.js';

export const taskModel = {
  async create(data) {
    const [r] = await pool.query(
      `INSERT INTO tasks
      (user_id, family_id, assigned_to, title, description,
      category, priority, frequency, due_date)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)` ,
      [
        data.user_id, data.family_id ?? null, data.assigned_to ?? null,
        data.title, data.description ?? null, data.category, data.priority,
        data.frequency, data.due_date,
      ]
    );
    return r.insertId;
  },

  async findById(id) {
    const [rows] = await pool.query('SELECT * FROM tasks WHERE id = ?', [id]);
    return rows[0] || null;
  },

  async markPendingReview(id, completedBy) {
    await pool.query(
      `UPDATE tasks SET status='pending_review', completed_by=? WHERE id=?`,
      [completedBy, id]
    );
  },

  async advanceRecurring(id, completedBy, frequency) {
    const unit = (frequency === 'daily' || frequency === 'weekly') ? 'DAY'
      : (frequency === 'monthly') ? 'MONTH' : 'YEAR';
    const n = frequency === 'weekly' ? 7 : 1;
    await pool.query(
      `UPDATE tasks
      SET due_date = DATE_ADD(due_date, INTERVAL ${n} ${unit}),
      completed_at = NOW(),
      completed_by = ?
      WHERE id = ?`,
      [completedBy, id]
    );
  },
};
```

2. Contrôleur avec logique de permission

```
// back/src/controllers/tasks.controller.js (extrait)
async function assertCanComplete(task, user) {
  if (!task.family_id) {
    if (task.user_id !== user.id) throw forbidden('Pas votre tâche');
    return { needsReview: false };
  }
  const member = await familyMemberModel.findByFamilyAndUser(task.family_id,
user.id);
  if (!member || member.status !== 'active') throw forbidden('Pas membre');
  if (member.role === 'parent') return { needsReview: false };
  if (task.assigned_to !== user.id) throw forbidden('Tâche non assignée');
  return { needsReview: true };
}

export const tasksController = {
  async complete(req, res) {
    const task = await taskModel.findById(Number(req.params.id));
    if (!task) throw notFound('Tâche introuvable');
    if (task.status === 'completed') throw badRequest('Déjà terminée');
    if (task.status === 'pending_review') throw badRequest('Déjà en attente');

    const { needsReview } = await assertCanComplete(task, req.user);

    if (needsReview) {
      await taskModel.markPendingReview(task.id, req.user.id);
    } else if (task.frequency === 'once') {
      await taskModel.markCompleted(task.id, req.user.id);
    } else {
      await taskModel.advanceRecurring(task.id, req.user.id, task.frequency);
    }
    res.json(await taskModel.findById(task.id));
  },
};
```

3. Validation systématique de l'entrée

```
// back/src/validators/task.validator.js (extrait)
import { CATEGORIES, PRIORITIES, FREQUENCIES } from '../config/constants.js';

const ISO_DATE = /^\\d{4}-\\d{2}-\\d{2}$/;

export function validateCreateTask(body) {
  const errors = {};
  const out = {};

  const title = body.title?.trim();
  if (!title || title.length > 200) errors.title = 'Titre invalide';
  else out.title = title;
```

```

    if (!CATEGORIES.includes(body.category)) errors.category = 'Catégorie invalide';
    if (!PRIORITIES.includes(body.priority)) errors.priority = 'Priorité invalide';
    if (!FREQUENCIES.includes(body.frequency)) errors.frequency = 'Fréquence
invalide';
    if (!ISO_DATE.test(body.due_date)) errors.due_date = 'Date invalide (YYYY-
MM-DD)';

    if (Object.keys(errors).length) throw badRequest('Champs invalides', errors);
    return { ...out, /* ...autres champs validés... */ };
}

```

Compétences mises en œuvre

- **Requêtes SQL préparées** (placeholders ?) → 0 risque d'injection
- Conception de **logique métier complexe** (3 chemins de complétion)
- **Séparation des responsabilités** : validator → controller → model
- Manipulation d'**ENUM MySQL** pour énumérations fortes
- Usage de **DATE_ADD/INTERVAL** SQL pour gestion native des dates récurrentes

Résultat

L'ensemble du module tâches est couvert par **21 scénarios de tests d'intégration** (CRUD, permissions, complétion, récurrence, validation parent) — tous passent. Le code est lisible et chaque fichier (route / controller / model / validator) tient sous 200 lignes.

Référence dans le code source

- [back/src/controllers/tasks.controller.js](#)
- [back/src/models/task.model.js](#)
- [back/src/validators/task.validator.js](#)
- [back/migrations/005_pending_review.sql](#)

♦ Exemple 2.3 — Génération de flux iCalendar (RFC 5545)

Projet : Reminder Famille **Période** : Mai 2026 **Stack** : Node.js 22, Express, bibliothèque ical-generator

Contexte

Le point fort technique du projet : permettre aux utilisateurs de recevoir des rappels natifs sur leur téléphone (iPhone, Android) sans installer d'application, en s'appuyant sur le standard **iCalendar (RFC 5545)** que tous les calendriers natifs comprennent.

J'ai conçu un service de génération iCal et trois endpoints exposant des flux différents (global, personnel, par famille).

Réalisation

1. Service de génération

```

// back/src/services/ical.service.js
import ical from 'ical-generator';

export function buildIcal({ ownerName, calendarName, calendarColor, tasks }) {
    const cal = ical({
        name: calendarName || `Reminder ${ownerName}`,

```

```

    prodId: { company: 'Reminder Famille', product: 'Reminder', language: 'FR' },
    timezone: 'Europe/Paris',
  });

  if (calendarColor) cal.x('X-APPLE-CALENDAR-COLOR', calendarColor);

  tasks.forEach((task) => {
    const event = cal.createEvent({
      id: `task-${task.id}@reminder-famille`,
      start: new Date(task.due_date),
      allDay: true,
      summary: task.title,
      description: task.description || '',
      categories: [{ name: task.category }],
    });

    // VALARM : notification système 1 jour avant l'échéance
    event.createAlarm({
      type: 'display',
      trigger: 86400, // secondes avant l'événement
      description: task.title,
    });
  });

  return cal.toString();
}

```

2. Routes (authentification par token URL)

```

// back/src/routes/calendar.routes.js
import { Router } from 'express';
import { calendarController } from '../controllers/calendar.controller.js';
import { asyncHandler } from '../utils/asyncHandler.js';

export const router = Router();

router.get('/:token',
  asyncHandler(calendarController.export));
router.get('/:token/perso.ics',
  asyncHandler(calendarController.exportPersonal));
router.get('/:token/family/:familyId.ics',
  asyncHandler(calendarController.exportFamily));

```

3. Controller

```

// back/src/controllers/calendar.controller.js (extrait)
async function userFromToken(token) {
  const cleaned = token.replace(/\.ics$/, '');
  const user = await userModel.findByCalendarToken(cleaned);
  if (!user) throw notFound('Calendrier introuvable');
  return user;
}

```

```

}

export const calendarController = {
  async export(req, res) {
    const user = await userFromToken(req.params.token);
    const tasks = await taskModel.listPendingForCalendar(user.id);
    const ics = buildIcal({ ownerName: `${user.first_name} ${user.last_name}`, tasks
  });
    res.set('Content-Type', 'text/calendar; charset=utf-8');
    res.set('Content-Disposition', `inline;
filename="reminder-${user.first_name}.ics"`);
    res.send(ics);
  },
  // ... exportPersonal et exportFamily similaires avec filtres SQL différents
};

```

Compétences mises en œuvre

- Étude et **application d'un standard** (RFC 5545 iCalendar)
- Intégration de **bibliothèque tierce** (ical-generator) via npm
- Conception d'une **authentification alternative** (token URL plutôt que JWT, car le calendrier externe ne peut pas envoyer de header Authorization)
- Configuration de **headers HTTP** appropriés (Content-Type , Content-Disposition)
- Génération de **3 flux filtrés** depuis une logique unique

Résultat

Les utilisateurs peuvent s'abonner depuis iPhone Calendar, Google Calendar ou Outlook. Les notifications déclenchées 24h avant chaque échéance par le système d'exploitation, sans aucune application Reminder Famille à installer. Testé sur iOS 17 et Android 14.

Aucune solution concurrente (Todoist, Trello, Notion) n'offre cette fonctionnalité sans abonnement payant.

Référence dans le code source

- [back/src/services/ical.service.js](#)
- [back/src/controllers/calendar.controller.js](#)
- [back/src/routes/calendar.routes.js](#)

4. Projet professionnel

4.1 Vision à court terme (6-12 mois post-formation)

À l'issue de la formation DWWM, je souhaite :

- **Consolider mes acquis** en continuant à développer Reminder Famille en production (déploiement Vercel + Fly.io, notifications email, internationalisation)
- **Rechercher une mission d'alternance ou un poste de junior développeur** dans une PME ou une startup, idéalement sur une stack Node.js/React proche de celle maîtrisée dans le projet
- **Continuer à publier** mes projets personnels sur GitHub pour étoffer mon portfolio

4.2 Vision à moyen terme (1-3 ans)

- Monter en compétence sur le **back-end pur** (Node.js avancé, PostgreSQL, Redis, Docker, CI/CD)
- Acquérir une expérience d'équipe (revue de code, méthodes Agile en équipe, gestion d'incidents)

- Envisager une **activité indépendante** complémentaire : développement de petites applications métier pour PME du secteur hôtellerie-restauration (que je connais particulièrement bien) — caisse enregistreuse, gestion de réservations, suivi de stock, etc.

4.3 Atouts pour cette reconversion

Mon parcours hors développement constitue un avantage différenciant :

Atout	Apport en développement
2 Masters (Moscou 2004 + France 2012)	Capacité d'analyse, recul stratégique, autoformation
10 ans en restauration (manager)	Gestion d'équipe, sens du service, gestion du stress
3 ans en casino (croupier)	Rigueur des process, attention soutenue, calculs rapides
Maturité (43 ans)	Recul, stabilité, capacité à prioriser et tenir des délais
Reconversion choisie	Motivation intrinsèque forte (pas un choix par défaut)

Je vise des structures qui valorisent l'autonomie, la rigueur et la relation client — des qualités que mes années de service salle et casino ont profondément ancrées.

5. Annexes

Annexe	Référence
Code source Reminder Famille	https://github.com/Vladimir08880888/reminder-famille
Dossier de Projet détaillé	../dossier-projet/dossier-projet.md
Suite de tests d'intégration (57 scénarios)	tests/integration.sh
Documentation API (30+ endpoints)	../dossier-projet/api-documentation.md
Document sécurité (OWASP)	../dossier-projet/securite.md
Diagrammes UML/Merise	../dossier-projet/diagrammes/
Screenshots de l'application	https://github.com/Vladimir08880888/reminder-famille/tree/main/annexes/screenshots

Document rédigé en mai 2026 pour la soutenance du Titre Professionnel Développeur Web et Web Mobile à l'AFPA Marseille.